

RegOS Expert-Mode Workflow Spec

1) What RegOS Is (Expert Perspective)

RegOS is a regulatory workflow copilot that connects three traditionally fragmented parties:

- **Citizen / Business Applicant** (non-expert, needs clarity and direction)
- **Consultant / Expert Preparer** (expert, needs speed + accuracy + automation)
- **Regulator / Reviewer** (authority, needs transparency + verification + audit trail)

RegOS exists to reduce friction, cycles, and rework by:

- turning dense regulatory code into **actionable guidance**
- producing **structured outputs** (forms, checklists, evidence packs)
- maintaining a **verifiable trail** of “what was done, why, and where in the code it’s supported”

Key idea: RegOS does **not replace expertise**. It “shepherds” experts to complete the work faster and with fewer misses, while keeping the regulator’s requirements traceable.

2) Persona-Based Success Definitions (What “Done” Means)

A) Citizen / Business Applicant (Non-expert)

Success = The user becomes meaningfully informed beyond what the regulator told them, without reading hundreds of pages of code, and exits with:

- a **plain-language understanding** of what they must do
- a **high-level checklist** of requirements
- optionally **direction to qualified help** (e.g., list of consultants, if data exists)
- “what conversation should I have next?” guidance

Failure =

- user remains confused, can’t determine next steps
- answers are too technical, ungrounded, or not explainable
- system asks expert-only questions the citizen can’t answer

B) Consultant (Expert User) — primary target in transcript

Success = The consultant can complete a compliant package **faster and more accurately**, with:

- rapid retrieval of relevant regulatory requirements
- **application/form fields filled** where possible (drafted, not “final legal truth”)
- a **requirements checklist** tailored to the case (QA/QC + completeness)
- a **provenance/audit trail**: who did what, when, how it maps to code, and what remains missing

- optional “workspace-like” behavior: working against a project folder / repository of facility documents

Failure =

- outputs aren’t reliably tied to the code (“hallucination risk”)
- missing key required elements → regulator rejects → rework cycles
- no traceability (“where in the code does it say that?” cannot be answered)
- system cannot use client-specific context (site specifics, prior permits, facility documents)

C) Regulator (Reviewer)

Success = Review becomes faster and less adversarial because the package includes:

- read-only visibility into compliance status and submissions
- **audit trail** of what requirements were satisfied and why
- ability to verify claims with **citations to the code + supporting evidence**
- clarity on what is missing or disputed

Failure =

- regulator can’t verify; must ask for rework repeatedly
- submissions are opaque or lack provenance
- no structured way to resolve disputes (consultant vs regulator vs applicant)

3) The RegOS “Mindset”: What RegOS Should and Should Not Do

What RegOS should do

- Act as a **regulatory interpreter** (code → requirements → checklist → outputs)
- Ask structured questions to gather missing context
- Generate drafts and structured artifacts (forms, checklists, evidence requirements)
- Maintain “source of truth” discipline: everything should be traceable

What RegOS should not do

- Invent requirements outside the applicable regulatory scope
- Pretend certainty without citations or traceability
- Replace engineering judgment where the expert must decide (RegOS guides; consultant owns the design/mitigation plan)

In the transcript, the “mind” comes from the consultant; the “rigid part” comes from the law/code.

4) RegOS Core Workflow (Generalized)

Step 1 — Identify the user goal and case type

RegOS begins by logging:

- **User's success definition** (what outcome they want)
- **Failure definition** (what would make this useless/wrong)
- Persona identification (citizen vs consultant vs regulator)

Step 2 — Constrain scope to the relevant regulatory domain

RegOS must operate within a defined **jurisdiction + code scope** (e.g., "Chapter 24 only" for EQCB).

- If user asks outside scope → RegOS should respond: **not applicable / out of boundary** and redirect.

Step 3 — Collect inputs (what data is needed to generate outputs)

Inputs fall into two categories:

A) Regulatory + Application Inputs (System-side)

- the governing code text
- application forms/templates
- known "as-is" process maps for that workflow

B) Facility / Case Context (User-side)

From the transcript, the consultant needs:

- site identity: **address, location, context** (e.g., sensitive area like wellfield)
- **previous permits / historical approvals**
- operational reality: what they do now, what's changing
- inspection info: **site visit notes, photos, field observations**
- violation/trigger: what the regulator flagged and why
- proposed mitigation ideas (expert-driven): e.g., containment, stormwater controls, operational restrictions, etc.

Important: Citizen vs Consultant questioning differs.

Citizens cannot answer "what fix do you propose?" Consultants can.

Step 4 — Generate outputs (draft artifacts)

For consultants, RegOS should produce:

- **Auto-filled drafts** of application forms (where possible)
- A **tailored requirements checklist** for submission completeness

- A **QA/QC provenance record** (“what was done, when, by whom, supported by which citations/evidence”)
- Evidence packaging guidance (what attachments are required, in what format)

For citizens, RegOS should produce:

- plain language requirements summary
- high-level checklist + next steps
- explanation-as-you-go (expand terms if asked)

For regulators:

- read-only verification views + audit trail
- structured report verification support
- transparency into what code provisions were used

Step 5 — Enable “transparent dialogue” loop

A core value in the transcript: reduce adversarial back-and-forth by making the system a shared reference layer.

When regulator says “missing X”:

- consultant can ask: “where in the code is that required?”
- citizen can see why work is needed and cost/time implications
- RegOS should keep the exchange grounded in citations and requirements.

5) Guardrails, Termination, and Exception Handling

Scope termination rule

RegOS should stop or refuse to provide authoritative workflow outputs when:

- the question is **outside the defined regulatory scope**
- required context is missing and user won’t provide it
- user requests an answer that requires expert site-specific engineering beyond available inputs (RegOS can list what’s needed, not fabricate it)

“Source of truth” policy

- Every requirement should be backed by:
 - code citation (and/or policy document)
 - or explicit statement: “I don’t have enough information; here’s what to provide.”

6) What “Never Changes” vs What Can Change

Stable (hard-programmable)

- the fact that the applicable code and workflow logic governs decisions
- roles/authority structure in that code (e.g., board overrides, director decisions) **until law changes**
- required submission components/checklist framework

Dynamic (case-specific)

- facility conditions, operations, proposed mitigation
 - evidence packages and engineering calculations
 - interpretation details that depend on site context
-

7) Product Implications for RegOS (What to Build)

If your colleague is asking “how should RegOS perform as an expert system,” the transcript implies these product capabilities:

1. Persona-aware questioning

- citizen-friendly vs consultant-grade prompts

2. Workspace + document repository access

- allow consultant to point RegOS to a “project folder” containing permits, drawings, prior approvals, inspections
- generate outputs grounded in that corpus

3. Automated checklist + QA/QC provenance

- produce submission completeness checklist
- record who validated what and when

4. Draft generation

- pre-fill forms/templates from gathered context
- generate structured sections (not “magic final,” but high-quality drafts)

5. Regulatory boundary enforcement

- hard stop outside scope; redirect rather than hallucinate

6. Transparency layer for regulator interaction

- citations and audit trail designed for review/verification